

Ejercicios con la clase `Nodo<T>`

Lista enlazada manual — Evidencias de funcionamiento (Java)

Se construyó manualmente, usando únicamente referencias entre objetos `Nodo<T>` (sin `ArrayList`, arreglos ni colecciones de Java), la lista enlazada de la imagen: **head -> A1 -> B -> C -> De -> Mc -> Zi**. Se realizaron todas las operaciones solicitadas: imprimir el estado inicial, imprimir solo el primer dato, imprimir el estado completo del último nodo, insertar "Fe" entre "De" y "Mc", insertar "Zz" al final, e insertar "Aa" al inicio — imprimiendo el estado de la lista después de cada cambio.

Estructura del proyecto

Archivo	Clase	Rol
Nodo.java	Nodo<T>	Clase genérica de nodo (tal cual se dio en clase, sin modificar)
Main.java	Main	Construcción manual de la lista + las 6 operaciones pedidas + métodos auxiliares de recor

Paso 1. Código fuente — Main.java (parte 1 de 2)

Construcción manual de la lista con referencias Nodo, y las operaciones 1 a 3 (estado inicial, dato del primer nodo, estado del último nodo).

```
package mx.unam.aragon.ico.edd.listas;

/**
 * Main.java
 * -----
 * Tarea 5: ejercicios con la clase Nodo<T>.
 *
 * Se construye manualmente, usando SOLO referencias entre
 * objetos Nodo (nada de ArrayList, arreglos ni colecciones de
 * Java para representar la lista), la lista enlazada de la
 * imagen:
 *
 * head -> Al -> B -> C -> De -> Mc -> Zi -> null
 *
 * y despues se realizan las inserciones que pide el enunciado.
 *
 * NOTA: el StringBuilder que se usa en imprimirListaSimplificada()
 * es solo para construir el TEXTO que se imprime en pantalla; no
 * representa ni almacena la lista (la lista sigue siendo, en todo
 * momento, una cadena de objetos Nodo enlazados por referencias).
 * -----
 */
public class Main {

    public static void main(String[] args) {

        // =====
        // 1) Construccion MANUAL de la lista de la imagen:
        // head -> Al -> B -> C -> De -> Mc -> Zi
        // =====
        Nodo<String> nodoAl = new Nodo<>("Al");
        Nodo<String> nodoB = new Nodo<>("B");
        Nodo<String> nodoC = new Nodo<>("C");
        Nodo<String> nodoDe = new Nodo<>("De");
        Nodo<String> nodoMc = new Nodo<>("Mc");
        Nodo<String> nodoZi = new Nodo<>("Zi");

        // Se enlazan uno con otro, tal como en DemoNodo:
        // head.setSiguiente(...) / .getSiguiente().setSiguiente(...)
        nodoAl.setSiguiente(nodoB);
        nodoB.setSiguiente(nodoC);
        nodoC.setSiguiente(nodoDe);
        nodoDe.setSiguiente(nodoMc);
        nodoMc.setSiguiente(nodoZi);
        // nodoZi.getSiguiente() queda en null: es el ultimo nodo.

        Nodo<String> head = nodoAl;

        // =====
        // 2) Estado inicial completo de la lista
        // =====
        System.out.println("=== 1) Estado inicial completo de la lista ===");
        System.out.println(head); // usa el toString() de Nodo (recursivo)
        System.out.print("Version simplificada: ");
        imprimirListaSimplificada(head);
        System.out.println();

        // =====
        // 3) Solo el dato del primer nodo
        // =====
        System.out.println("=== 2) Dato del primer nodo ===");
        System.out.println(head.getDato());
        System.out.println();

        // =====
        // 4) Estado completo del nodo en la ultima posicion
        // =====
        System.out.println("=== 3) Estado completo del ultimo nodo ===");
        Nodo<String> ultimoNodo = obtenerUltimoNodo(head);
        System.out.println(ultimoNodo);
        System.out.println();
    }
}
```

Paso 1 (continuación). Código fuente — Main.java (parte 2 de 2)

Las inserciones ("Fe", "Zz", "Aa") y los métodos auxiliares de recorrido (buscarNodo, obtenerUltimoNodo, imprimirListaSimplificada), todos implementados solo con referencias Nodo.

```
nodoDeEncontrado.setSiguiente(nodoFe);           // De -> Fe

System.out.println("Nuevo estado de la lista:");
System.out.println(head);
System.out.print("Version simplificada: ");
imprimirListaSimplificada(head);
System.out.println();

// =====
// 6) Insertar "Zz" al final de la lista
// =====
System.out.println("=== 5) Insertar \"Zz\" al final de la lista ===");
Nodo<String> nodoZz = new Nodo<>("Zz");
Nodo<String> ultimoActual = obtenerUltimoNodo(head);
ultimoActual.setSiguiente(nodoZz);

System.out.println("Nuevo estado de la lista:");
System.out.println(head);
System.out.print("Version simplificada: ");
imprimirListaSimplificada(head);
System.out.println();

// =====
// 7) Insertar "Aa" al inicio de la lista
// =====
System.out.println("=== 6) Insertar \"Aa\" al inicio de la lista ===");
Nodo<String> nodoAa = new Nodo<>("Aa");
nodoAa.setSiguiente(head); // Aa -> (antiguo head, Al)
head = nodoAa;           // Aa se vuelve el nuevo primer nodo

System.out.println("Estado final de la lista:");
System.out.println(head);
System.out.print("Version simplificada: ");
imprimirListaSimplificada(head);
}

// -----
// Métodos auxiliares: recorren la lista SOLO con referencias Nodo,
// sin usar arreglos, ArrayList ni ninguna colección de Java.
// -----

/** Recorre la lista desde 'inicio' hasta encontrar el nodo cuyo dato sea 'valorBuscado'. */
private static <T> Nodo<T> buscarNodo(Nodo<T> inicio, T valorBuscado) {
    Nodo<T> actual = inicio;
    while (actual != null) {
        if (actual.getDato().equals(valorBuscado)) {
            return actual;
        }
        actual = actual.getSiguiente();
    }
    return null; // no se encontro
}

/** Recorre la lista desde 'inicio' hasta el nodo cuyo siguiente es null (el ultimo). */
private static <T> Nodo<T> obtenerUltimoNodo(Nodo<T> inicio) {
    Nodo<T> actual = inicio;
    while (actual.getSiguiente() != null) {
        actual = actual.getSiguiente();
    }
    return actual;
}

/** Imprime la lista como "Al -> B -> C -> ..." en vez del toString anidado de Nodo. */
private static <T> void imprimirListaSimplificada(Nodo<T> inicio) {
    StringBuilder texto = new StringBuilder();
    Nodo<T> actual = inicio;
    while (actual != null) {
        texto.append(actual.getDato());
        if (actual.getSiguiente() != null) {
            texto.append(" -> ");
        }
    }
}
```

Paso 2. Compilación

Se listan los 2 archivos fuente del paquete **mx.unam.aragon.ico.edd.listas** y se compilan con **javac**, sin errores.

```
$ ls src/mx/unam/aragon/ico/edd/listas/  
Main.java  Nodo.java  
  
$ javac -d bin -encoding UTF-8 src/mx/unam/aragon/ico/edd/listas/*.java  
Compilacion exitosa.  
█
```

Paso 3. Ejecución completa

Salida de `java mx.unam.aragon.ico.edd.listas.Main`, mostrando el estado de la lista en cada uno de los 6 pasos solicitados. Cada línea '`Nodo{...}`' es el `toString()` real de la clase `Nodo` (anidado, recursivo); la línea '`Version simplificada`' es una representación adicional en formato '`A -> B -> C`' para facilitar la lectura.

```
$ java mx.unam.aragon.ico.edd.listas.Main
=== 1) Estado inicial completo de la lista ===
Nodo{dato=Al, siguiente=Nodo{dato=B, siguiente=Nodo{dato=C, siguiente=Nodo{dato=De, siguiente=Nodo{dato=Mc, siguiente=Nodo{dato=Zi, siguiente=null}}}}}}
Version simplificada: Al -> B -> C -> De -> Mc -> Zi

=== 2) Dato del primer nodo ===
Al

=== 3) Estado completo del ultimo nodo ===
Nodo{dato=Zi, siguiente=null}

=== 4) Insertar "Fe" entre "De" y "Mc" ===
Nuevo estado de la lista:
Nodo{dato=Al, siguiente=Nodo{dato=B, siguiente=Nodo{dato=C, siguiente=Nodo{dato=De, siguiente=Nodo{dato=Fe, siguiente=Nodo{dato=Mc, siguiente=Nodo{dato=Zi, siguiente=null}}}}}}}}
Version simplificada: Al -> B -> C -> De -> Fe -> Mc -> Zi

=== 5) Insertar "Zz" al final de la lista ===
Nuevo estado de la lista:
Nodo{dato=Al, siguiente=Nodo{dato=B, siguiente=Nodo{dato=C, siguiente=Nodo{dato=De, siguiente=Nodo{dato=Fe, siguiente=Nodo{dato=Mc, siguiente=Nodo{dato=Zi, siguiente=Nodo{dato=Zz, siguiente=null}}}}}}}}}}
Version simplificada: Al -> B -> C -> De -> Fe -> Mc -> Zi -> Zz

=== 6) Insertar "Aa" al inicio de la lista ===
Estado final de la lista:
Nodo{dato=Aa, siguiente=Nodo{dato=Al, siguiente=Nodo{dato=B, siguiente=Nodo{dato=C, siguiente=Nodo{dato=De, siguiente=Nodo{dato=Fe, siguiente=Nodo{dato=Mc, siguiente=Nodo{dato=Zi, siguiente=Nodo{dato=Zz, siguiente=null}}}}}}}}}}}}
Version simplificada: Aa -> Al -> B -> C -> De -> Fe -> Mc -> Zi -> Zz
```